CSE461

Introduction to Robotics Lab

Lab No. : 04
Group : 01
Section : 06
Semester : Summer_2025

Group members :

Sayeba Nasir	22201354
B M Rauf	22201782
Azmari Sultana	22201949

Submitted Date: 15-08-2025
Submitted to - CSDM

1. **Objectives:**

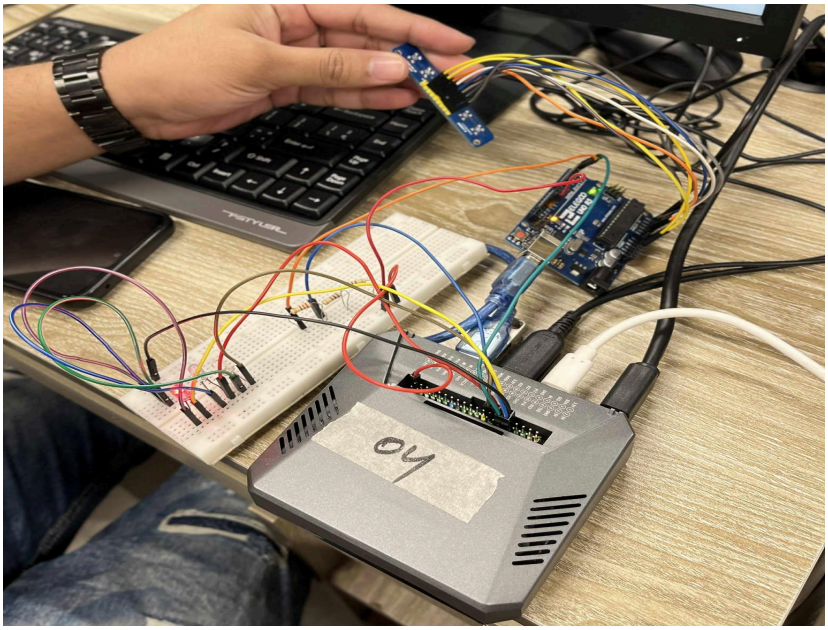
   The goal of this task was to add and set up three LEDs with Raspberry Pi and make them light up based on the movement status coming from the Arduino UNO. So, this task was to expand the Arduino and Raspberry Pi so that it doesn't only just show movement status on the LCD but can also work with physical components like here three LED's were used for quick direction indication as movement status. Arduino Uno reads from the six array IR sensors then decides the direction (LEFT, RIGHT, FORWARD, UNCERTAIN) and sends that decision to the Raspberry Pi over UART.

2. **Equipments:**

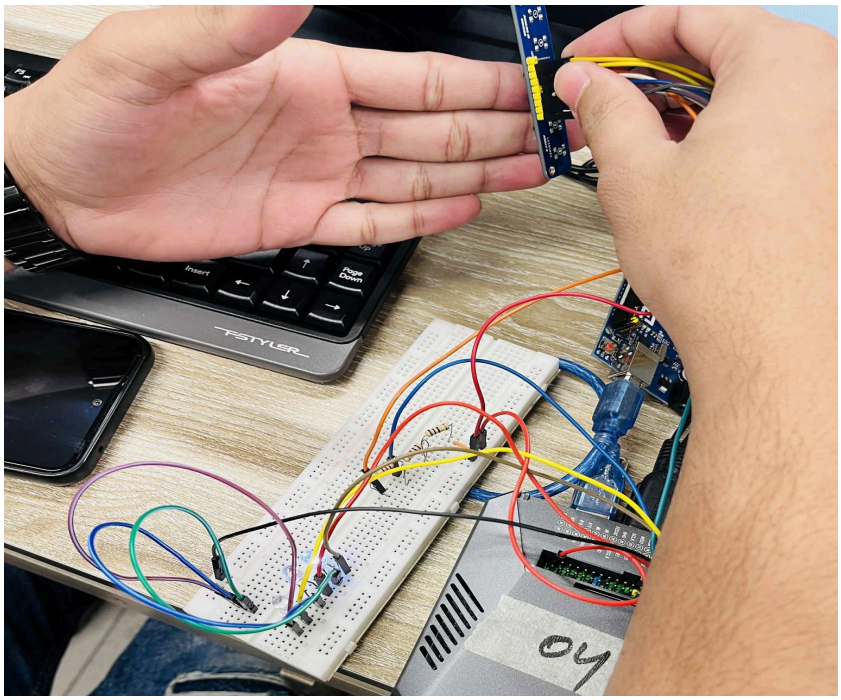
   - Raspberry Pi
   - Arduino Uno R3
   - Analog 6-array IR sensor
   - 3 LEDs
   - Total 3 of 220 Ω resistors
   - Breadboard
   - Jumper wires

3. **Experimental Setup:**

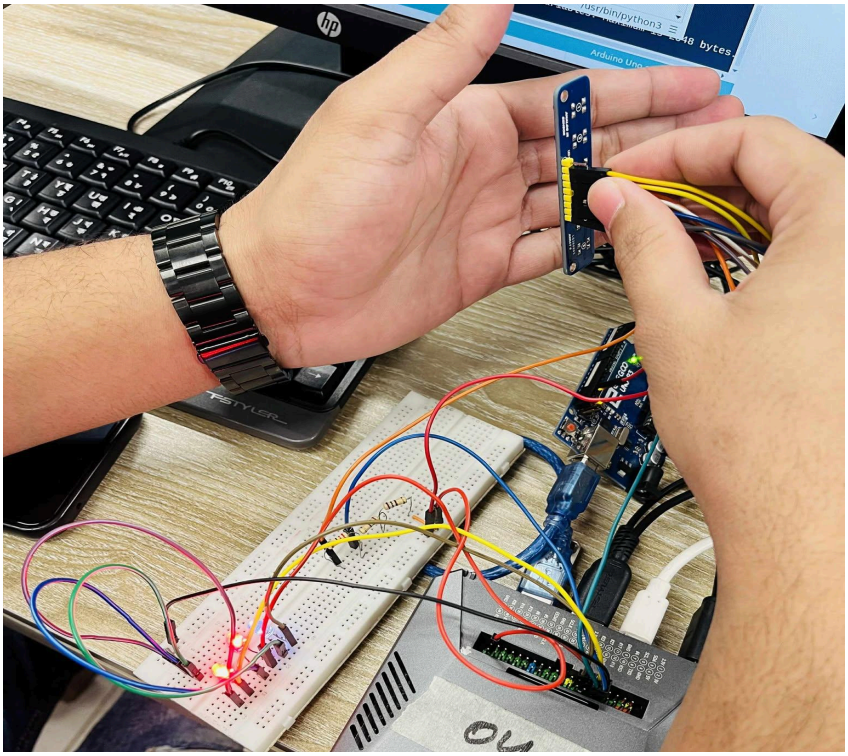
   In this experiment, we built on the previous Arduino–Raspberry Pi setup that we used for our previous tasks and we added three LEDs to the Raspberry Pi so that the movement status could be seen instantly by looking at the LEDs. We connected Arduino Uno to an Analog 6-array IR sensor, with its VCC wired to the Arduino's 5V pin, GND to Arduino ground, and each of the six sensor outputs going to analog pins A0 through A5. The Arduino processed the received sensor readings and determined whether the movement or obstacle was by LEFT, RIGHT, FORWARD and then sent that decision to the Raspberry Pi. The three LEDs were connected directly to the Raspberry Pi's GPIO pins using three 220 Ω resistors - the left LED on GPIO17 (Pin 11), the middle LED on GPIO27 (Pin 13), and the right LED on GPIO22 (Pin 15), with all cathodes connected to the Raspberry Pi's ground. The Python code used a library : "`gpiozero`" to control these LEDs, turning only the left LED on when the Arduino reported "LEFT," only the right LED on for "RIGHT," all three LEDs on for "FORWARD," and turning all LEDs off for any other message such as "UNCERTAIN". This setup meant that every time the Arduino sent a movement status, the LEDs updated instantly, giving a real-time update.



Picture 1: Left LED turn on after detecting sensor status is LEFT



Picture 2: Right LED turn on after detecting sensor status is RIGHT



Picture 3: All LED's turned on after detecting sensor status is FORWARD

4. **Code:**

Arduino Code:

```
#define NUM_SENSORS 6
int sensorPins[NUM_SENSORS] = {A0, A1, A2, A3, A4, A5};
int sensorValues[NUM_SENSORS];

void setup() {
 Serial.begin(9600);
 for (int i = 0; i < NUM_SENSORS; i++) {
   pinMode(sensorPins[i], INPUT);
 }
}

void loop() {
 for (int i = 0; i < NUM_SENSORS; i++) {
   sensorValues[i] = analogRead(sensorPins[i]);
 }
```

```
 int threshold = 500;
 int binary[NUM_SENSORS];
 for (int i = 0; i < NUM_SENSORS; i++) {
   binary[i] = (sensorValues[i] > threshold) ? 0 : 1;
 }

 if (binary[0] == 1 && binary[1] == 1 && binary[2] == 1 && binary[3] == 1 && binary[4] == 1
&& binary[5] == 1) {
   Serial.println("STOP");
 }
 else if (binary[0] == 0 && binary[1] == 0 && binary[2] == 0 && binary[3] == 0 && binary[4]
== 0 && binary[5] == 0) {
   Serial.println("FORWARD");
 }
 else if (binary[0] == 0 || binary[1] == 0) {
   Serial.println("LEFT");
 }
 else if (binary[4] == 0 || binary[5] == 0) {
   Serial.println("RIGHT");
 }
 else {
   Serial.println("UNCERTAIN");
 }

 delay(200);
}
```

RaspberryPI Code:

```
import serial
from gpiozero import LED

# GPIO pins
LED_LEFT_PIN  = 17
LED_MID_PIN   = 27
LED_RIGHT_PIN = 22

left  = LED(LED_LEFT_PIN)
mid   = LED(LED_MID_PIN)
right = LED(LED_RIGHT_PIN)

def all_off():
   left.off(); mid.off(); right.off()

def on_left():
   left.on();  mid.off(); right.off()
```

```
def on_right():
  right.on(); mid.off(); left.off()

def on_forward():
  left.on(); mid.on(); right.on()

ser = serial.Serial('/dev/ttyACM0', 9600, timeout=1)
ser.reset_input_buffer()

try:
  while True:
    line = ser.readline().decode('utf-8', errors='ignore').strip().upper()
    if not line:
      continue
    print(f"Arduino: {line}")
    if line == "LEFT":
      on_left()
    elif line == "RIGHT":
      on_right()
    elif line == "FORWARD":
      on_forward()
    else:
      all_off()
except KeyboardInterrupt:
  all_off()
  ser.close()
```

5. **Results (Output of the experiment):**

In this task, we connected three LEDs to the Raspberry Pi and made them turn on or off depending on what the Arduino's IR sensors detected. The Arduino read the sensor values, decided whether the movement should be LEFT, RIGHT, FORWARD, or something else, and then sent this decision to the Raspberry Pi through a UART connection. Our Raspberry Pi code worked for these messages and controlled the LEDs like: "LEFT" : Only the left LED turned ON ; "RIGHT" : Only the right LED turned ON ; "FORWARD": All three LEDs turned ON together and for others or anything else all LEDs stayed OFF. When we tested it, the LEDs reacted almost instantly to the sensor changes. This showed that the communication between the Arduino and Raspberry Pi was working perfectly and the LEDs were correctly showing the accurate movement status and results.

6. Discussions :

This experiment showed and teached us how the Raspberry Pi that we were using for other tasks before can also be used to control even physical components like LEDs based on the data it receives from another device like Arduino. The UART communication worked smoothly and there was no noticeable delay between sensor detection and LED response. The setup was simple though we faced issues with the wires so we had to change them again and again as somehow some connections were getting loose so it took us some time but we solved the issues promptly and also it was indeed an effective experiment and it also showed how multiple devices can work together in real time with such an easy setup. Overall, this task was a good example of combining hardware control with serial communication to help us while creating interactive, responsive systems.